Company Brain: Technical Annexes

Alexandru Rada · 11 October 2026 · Edition 1.1

Annex A. Record and boundary contracts

A Task record identifies the objective, project, authority, acceptance, dependencies, permitted effects, output destination, execution limit and state. A ContextBundle identifies selected sources by version and purpose. An Attempt identifies the executor, input snapshot, operation identity, observed tool responses and artifact. An Evaluation identifies the exact artifact, criterion, method, result and evaluator. A MemoryRecord connects a retained consequence to those records. A SkillVersion describes applicability, prerequisites, procedure, checks, counterexamples and dependencies.

These are proposed contracts rather than a claim that every field has a publicly reproduced implementation. Their minimum semantic requirement is that a reviewer can distinguish requested work, performed work, checked output and accepted reuse. The contracts can be represented in different storage systems without changing those distinctions.

Boundary invariants are as follows. A dependent result is not current merely because its text is unchanged. A candidate procedure is not approved because it has been indexed. A retry does not acquire new authority. An evaluation of one artifact does not validate a replacement artifact. A completed tool request does not establish task acceptance. An archived record remains historical evidence even when excluded from current retrieval.

Annex B. Reproduction of the public fixture

The companion file examples/demo.py uses Python's standard library, dataclasses and unittest. It constructs synthetic Source and Result objects in memory. From the repository root, run:

python examples/demo.py

The expected terminal result is five tests executed successfully. The cases exercise current retrieval, changed-source rejection, project mismatch, candidate exclusion and artifact modification. No credentials, network requests, model calls or external data are required. A nonzero exit status indicates that at least one expectation failed or execution encountered an error.

The successful control returns the stored result text. Each negative case expects no reusable result. The source-version case changes the source while retaining the old result version. The project case asks from another project. The candidate case sets accepted to false. The integrity case changes result text without changing its recorded digest. These interventions isolate the corresponding conditions in the retrieval function.

The fixture has important limits. Its objects do not persist across process restarts. Project strings are caller-provided and offer no authentication. It does not evaluate factual accuracy, retrieve documents, run a model, schedule a workflow or implement concurrency. A passing suite establishes only the five expressed mechanical expectations. The public release's verification record reports the actual interpreter and test result.

Annex C. Proposed adversarial and recovery cases

Unsupported evidence: supply a task for which no eligible source exists. The system should identify the gap without inventing a supporting record. Evaluation inspects both the answer and its claimed source identities.

Stale dependency: change a governing source after context selection. The system should prevent unexamined acceptance of the dependent result and preserve the original attempt. Evaluation compares the execution and acceptance snapshots.

Duplicate delivery: deliver the same operation identity twice. The host should reconcile the existing effect according to the provider's actual contract. A provider without reliable reconciliation yields an uncertain outcome rather than a fabricated exactly-once guarantee.

Interrupted execution: stop a worker after an observable side effect and before local completion. Recovery should inspect the original effect before retrying. This requires an isolated test service; the public in-memory fixture does not implement it.

Misleading reflection: provide a fluent candidate lesson contradicted by the underlying result. The curator should retain the discrepancy and reject or narrow the lesson rather than promote it because of its presentation.

Overgeneralisation: derive a procedure from one successful example, then test a different subject and a counterexample. A case-specific constant, voice or permission must not silently become a universal instruction. Transfer is assessed by task results, not by the presence of an abstract summary.

Annex D. Experimental specification

The primary comparison is facts plus validated procedures against the same facts without procedures. A current-task-only arm measures the additional contribution of factual retention. All arms share a fixed model, tools, maximum budget and predefined rubric. Each receives isolated state. Development and held-out tasks are separated by family; equivalent instances remain in the same split.

The analysis unit is a task outcome, with repeated runs and task-family dependence retained in the dataset. A proposed family-clustered bootstrap estimates uncertainty in paired acceptance differences. The sample size is selected from a pilot and a stated minimum meaningful effect before the held-out evaluation. No sample-size adequacy is claimed without that calculation.

The run record contains task family, condition, order, model and tool configuration, source versions, elapsed time, active human correction time, resource use, outcome, failure category and evaluation evidence. Unsupported claims and stale-source use are reported separately from overall acceptance. Failed attempts remain in cost accounting. An undefined cost-per-accepted-result ratio is never replaced with zero.

Online correction is a separate protocol. It specifies when updates are permitted, which evidence can change a procedure, and how later tasks are selected. Navigation and packaging ablations vary their target mechanism while holding other conditions fixed. This prevents gains from a larger model or richer corpus being attributed to a map or memory policy.

Annex E. Claim-to-evidence map

Worked examples expose reusable relations: supported as a human-learning finding by Company Brain reference [1]; agent transfer remains an engineering hypothesis tested on separate tasks.

External information can influence fixed-model generation: mechanism supported by reference [3]; Company Brain's particular update policy is a proposed integration.

Relevant information can be used differently according to context position: supported in the tasks studied by reference [4]; no universal context-length claim follows.

Retained feedback, insights and skills can guide later agent activity: research mechanisms in references [5–9]; their numerical results are not Company Brain results.

Source eligibility can reject an obsolete result: directly demonstrated by the public version-mismatch fixture, within its in-memory scope.

Better transfer, lower cost and reduced correction effort: proposed outcomes requiring the controlled experiments above. They are not measured release claims.

Annex F. Artifact interpretation

The MAPS screenshot in the main paper is an observed interface view with detailed item labels hidden. The other maps are synthetic explanatory diagrams. Neither proves that a provider, schedule or worker executed. Their purpose is to make relations inspectable; receipts and evaluations establish particular observed outcomes.

The release contains public papers, illustrations and a narrow educational example. It contains no private conversations, operational prompt archive, provider credentials, production memory database or licensed vendor package. This boundary makes the argument inspectable without presenting the author's private system as an independently reproduced experimental artifact.